VAKRATRON SYSTEMS

WHITEPAPER · ENTERPRISE LLM

Running LLMs in the Enterprise

Choosing between API and self-hosted models, governing them, and keeping costs predictable

Written for
CIOs, IT heads, enterprise and AI architects, security, risk and data protection teams

Version 1.0, October 2026
vakratronsys.com

Contents

01 Executive summary
02 Why LLM projects stall
03 The building blocks, explained properly
04 API, private endpoint, self-hosted or hybrid
05 Choosing: use cases first, then models
06 Reference architecture
07 Sizing and cost
08 Security, data protection and the DPDP Act
09 Operating it: evaluation, monitoring and change
10 A practical roadmap
11 Ten common mistakes
12 Enterprise LLM readiness checklist (36 points)
13 Glossary
14 About Vakratron Systems

What you will take away

- A simple way to triage use cases by value, risk and how easily answers can be checked
- Public API, private endpoint, self-hosted and hybrid compared on data, quality and cost
- A cost crossover formula: the daily token volume at which your own GPUs pay off
- How to choose and test models on your own work, not on leaderboards
- When to prompt, when to use RAG and when fine-tuning is worth it
- An LLM gateway design for routing, PII redaction, budgets and audit, with DPDP Act points
- A 36-point readiness checklist for platform, security and operations

01

Executive summary

Most organisations already use large language models. Often the first use is unofficial: staff pasting documents into free chat tools on personal accounts. The question for IT leadership is no longer whether to use LLMs, but how: which tasks, which models, where they run, what data they may see, and how anyone knows the answers are good enough and the bill is under control.

This paper sets out a practical approach for the IT leaders who decide and the security, risk and data protection teams who sign off.

Five things to take away

1. **Start from tasks, not models.** Pick a few frequent, well-defined tasks where answers can be checked. A narrow task done well beats "AI everywhere" done badly.
2. **Test on your own examples.** A test set of 100 to 300 real cases, with answers agreed by the people who do the work, is the single most valuable asset in an LLM programme.
3. **Self-hosting is a data and volume decision.** It pays off on cost only above a daily token volume you can calculate. Below that, data rules are the only good reason.
4. **Put one gateway in front of every model.** Sign-in, redaction, routing, budgets and logging in one place, so models can change without changing every application.
5. **Treat models like any other production change.** New models and prompts go through test, approval and gradual rollout, with a quick way back.

02

Why LLM projects stall

LLM pilots are easy to start and hard to finish. A demo on ten hand-picked questions impresses in a week; handling thousands of messy, real requests safely and at a known cost takes discipline most pilots skip. The same problems recur:

What we find	What happens
No approved option for staff	People use whatever is free. Customer data and internal documents leave the company without anyone deciding they should.
Pilot judged on easy questions	Real users ask vague, multi-part and out-of-scope questions. Quality drops, and nobody measured it before go-live.
Every app calls the model directly	Keys are scattered in code, nobody can see total usage, and changing the model means changing every application.
Long prompts on the largest model	Whole documents are sent with every request to the most expensive model. Monthly cost becomes a board-level question.
Fine-tuning to teach facts	Months go into training a model to “know” policies that change every quarter, when the real need was to look them up.
No owner after go-live	Quality drifts after a model update or a document change, and complaints are the first warning.

> **In short:** most LLM failures are not model failures. They come from unclear tasks, untested quality, uncontrolled access to data and unmanaged cost.

03

The building blocks, explained properly

Tokens and context

Models read and write in **tokens**, pieces of words. In English one token is roughly three quarters of a word, so 1,000 words is about 1,300 tokens. Hindi and other Indian languages in their own scripts often take two to four times as many tokens for the same meaning on many models, which raises cost and fills the context faster. The **context window** is how many tokens the model considers at once. Output tokens usually cost three to five times more than input.

Model size

Small open models of around 8 billion parameters (8B) are fast and cheap, and handle extraction, classification and routing well. Models around 70B handle most drafting, summarising and question answering. The largest commercial models lead on complex reasoning and long documents.

Serving

An open model runs on a **model server** such as vLLM or NVIDIA Triton with TensorRT-LLM. Three techniques let one server handle hundreds of users rather than a few:

- **Continuous batching:** requests join and leave the batch at every step, so the GPU stays busy instead of waiting for the slowest request.

- **Paged KV cache:** the memory that holds each conversation's context is allocated in small blocks, so many more requests fit on one GPU.
- **Prefix caching:** a long system prompt or a shared document is processed once and reused across requests.

Two numbers describe the user experience: **time to first token** (how long before the answer starts, ideally under one to two seconds for chat) and **output tokens per second per user** (reading speed is about 10 to 15; 30 or more feels instant).

Quantisation

Quantisation stores weights in fewer bits. FP8 halves memory against 16-bit with little measurable quality loss on most tasks, and runs natively on H100, H200 and L40S GPUs. 4-bit formats such as AWQ halve it again but can hurt accuracy, especially in Indian languages. Re-run the test set on any quantised model.

Prompting, RAG or fine-tuning

These are often confused. They solve different problems and are usually used together:

The problem	Try first	Why
The model does not know our policies, products or cases	RAG: look up the relevant passages and give them to the model	Facts change. Looking them up at question time keeps answers current and lets you show sources.
Answers are in the wrong format or tone	Better prompts with two or three good examples, then fine-tuning	Examples in the prompt fix most format problems. Fine-tune only if it still drifts.
A narrow task repeated thousands of times a day	Fine-tune a small model	A small fine-tuned model can match a large general one on one task at a fraction of the cost.
Specialised language, such as legal or internal jargon	RAG plus a glossary, then fine-tuning if needed	Retrieval supplies the facts; fine-tuning helps the model use the terms naturally.

Fine-tuning today usually means **LoRA**: training a small set of extra weights rather than the whole model. It needs hundreds to a few thousand good examples, fits on one 8-GPU server for a 70B model, and usually has to be repeated when the base model changes.

04

API, private endpoint, self-hosted or hybrid

There are four ways to give the organisation access to LLMs. They differ in where data goes, which models are available, how cost behaves as usage grows, and how much the IT team has to run.

Option	Where data goes	Models	Cost behaviour	What you run
Public API	To the provider, under its enterprise terms	Strongest commercial models	Pure per-token; grows in line with use	Almost nothing
Private cloud endpoint	Stays in your cloud tenancy and chosen region	Commercial models offered in that region	Per-token or reserved capacity	Cloud networking, keys, access
Self-hosted open model	Never leaves your network	Open-weight models (Llama, Mistral, Qwen, Gemma and others)	Mostly fixed; cheap per token at high use	GPUs, model servers, platform, upgrades
Hybrid behind a gateway	Decided per request by data class	Both	Fixed base plus per-token for the rest	Gateway and a smaller GPU estate

Public API

The fastest start and the strongest models. The questions are contractual: whether prompts are retained or used for training, where they are processed, and what cost looks like at ten times today's volume. Enterprise agreements usually exclude training on your data; consumer terms often do not.

Private cloud endpoint

Commercial models deployed inside your own cloud account, in an Indian region where available. Data stays within your tenancy, under controls you already use. Check which models each region offers.

Self-hosted open model

An open-weight model on your own GPUs. Data never leaves infrastructure you control, versions change only when you decide, and it can run air-gapped. In exchange you run GPUs, serving and upgrades, and open models may trail the best commercial ones on the hardest tasks. Read each licence: some restrict use above a user count.

Hybrid behind a gateway

Most organisations end up here: confidential and personal data goes to self-hosted models, while redacted public and internal content may use a commercial model. A gateway makes the choice per request.

05

Choosing: use cases first, then models

Triage the use cases

Score each candidate on frequency, value, data sensitivity and how easily a person can check the output. The best first projects are frequent, valuable and easy to check.

Use case type	Examples	Risk	Typical first approach
Draft for a person to edit	Email replies, meeting notes, first drafts of reports	Low: a person reviews	API or private endpoint; light controls
Answer from documents	Policy and product questions, IT and HR help	Medium: wrong or leaked answers	RAG with permissions and sources
Extract and classify	Claim forms, invoices, tickets, KYC documents	Medium: errors flow into systems	Small or fine-tuned model, confidence thresholds
Customer-facing conversation	Service chat, contact centre assist	High: brand and conduct risk	Assist an agent first, then limited self-service
Decisions about people	Credit, claims, hiring, eligibility	Very high: fairness and regulation	Decision support only; a person decides

Five questions that decide hosting

1. **May this data leave our network?** Classify the data each use case touches. If personal, customer financial or restricted government data is involved, sector rules and your own policy may decide for you.
2. **What is the daily token volume, now and in a year?** Compare it with the crossover point in the sizing section. Below it, APIs are cheaper.
3. **Does an open model pass the test set?** If only a frontier commercial model meets the quality bar, self-hosting the task is not yet an option.
4. **Is there a hard latency, availability or air-gap requirement?** Plants, defence and some government networks cannot depend on an internet path.
5. **Can we run GPUs well?** If no team owns drivers, serving and capacity, budget for one or use a managed private option.

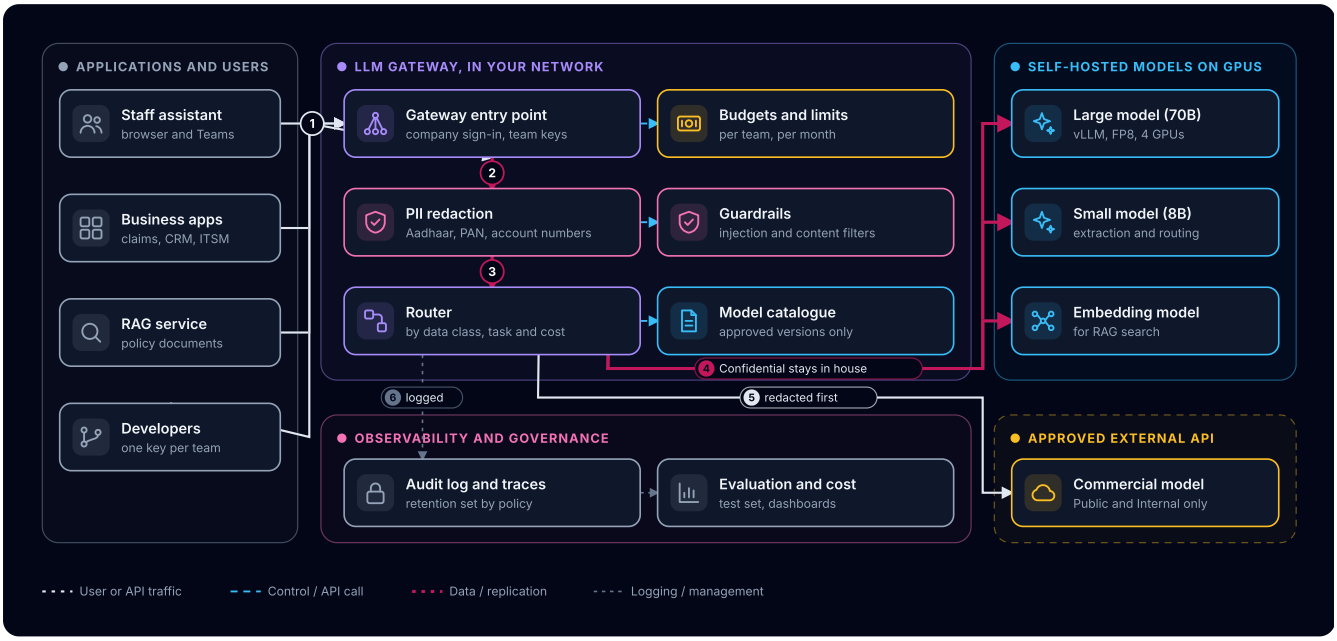
Testing models on your own work

Collect 100 to 300 real examples with the answer a good employee would give, including messy and out-of-scope cases, where models differ most. Define “good”: correct facts, right format, and “I don’t know” when it should. Score two or three candidates automatically and by reviewers, then weigh cost and speed. A model 3 percent better but ten times dearer is rarely right. Keep the test set: a new model version can then be judged in an afternoon.

06

Reference architecture

The diagram shows a hybrid platform built around one LLM gateway. Applications never call a model directly: the gateway knows who is asking, cleans the request, picks a model, enforces budgets and records what happened.



Numbered flows: (1) every application and user calls one gateway endpoint, authenticated by company sign-in or a team key, (2) each request passes PII redaction and guardrails, (3) the router reads the data class tag and task type, (4) Confidential and Restricted requests go only to self-hosted models, with the small model handling extraction and routing, (5) Public and Internal requests may go to an approved commercial model after redaction, (6) every request, model choice and cost is logged for audit and evaluation.

What the gateway does

Function	In practice
Identity and access	Company sign-in for people, one key per team for applications. Groups decide which models and data classes each team may use.
Redaction	Personal data is masked before any model sees it: names, phone numbers, Aadhaar, PAN, bank account and IFSC codes, using Microsoft Presidio plus rules for Indian identifiers.
Routing	By data class, task and cost: small model for extraction, larger model for drafting, commercial model only for permitted classes. Fallback to a second model if one is down.
Budgets and limits	Monthly budget per team with alerts at 50, 80 and 100 percent, a cap on tokens per request, and rate limits so one runaway job cannot starve everyone else.
Logging	Who asked, which model answered, tokens, cost, latency and guardrail decisions. Prompt text is kept only as long as policy allows.

> **Common tools:** LiteLLM or an API gateway with LLM routing, vLLM on Kubernetes with the NVIDIA GPU Operator, Presidio and NeMo Guardrails, and Langfuse or OpenTelemetry for traces. All are replaceable; the pattern is what matters.

07

Sizing and cost

The cost crossover

An API costs a fixed amount per token. A self-hosted model costs roughly the same each month whether it is busy or idle. The volume at which the two are equal is the crossover:

```
Crossover tokens per day = monthly self-hosted cost (₹) × 1,000,000 ÷ (active days per month × blended API price per million tokens (₹))
```

The blended price mixes input and output at your real ratio. For a frontier-class model at ₹250 per million input tokens and ₹1,250 per million output, a 3:1 input to output mix blends to (3 × 250 + 1,250) ÷ 4 = ₹500 per million. Mid-tier commercial models are often a quarter of that or less.

What self-hosting costs per month

Planning ranges for one 8-GPU H100 server running a 70B-class model in FP8 as two 4-GPU replicas:

Item	Basis	Monthly, planning range
Server, networking and storage share	₹2.6 to 3.2 crore, over 48 months	₹5.4 to 6.7 lakh
Power and cooling	About 10.2 kW × 1.5 PUE × 730 hours = 11,200 kWh at ₹8 to 10	₹0.9 to 1.1 lakh
Rack space in a data centre or colocation	One high-density rack position	₹0.8 to 1.5 lakh
Support, software and spares	Hardware support, enterprise software where used	₹0.5 to 1.0 lakh
Platform engineering	About 1.5 people across GPUs, serving and gateway	₹2.5 to 3.5 lakh
Total		**₹10 to 14 lakh, midpoint ₹12 lakh**

With ₹12 lakh a month, 22 working days and a frontier-class API at ₹500 per million tokens, the crossover is 12,00,000 × 1,000,000 ÷ (22 × 500), or about **110 million tokens a day**. Against a mid-tier API at ₹120 per million it rises to about 450 million tokens a day, which is more than one server can handle. Self-hosting wins on cost only when the task needs a large model and volume is high.

GPU memory

```
GPU memory needed ≈ model weights + (KV cache per token × context length × concurrent requests) + 10 to 20% overhead
```

A 70B model’s weights need about 140 GB in 16-bit, 70 GB in FP8 and 35 to 40 GB in 4-bit; its KV cache is about 0.33 MB per token. On four 80 GB H100s (320 GB), FP8 weights take 70 GB and overhead about 40 GB, leaving roughly 210 GB for cache: about 640,000 tokens, or around 80 concurrent requests at 8,000 tokens of context each.

A worked example

A financial services firm plans two uses. A staff assistant: 1,500 daily users × 12 questions × 4,000 tokens (3,500 in, 500 out) = 72 million tokens a day. Loan file summaries: 1,000 files × 30,000 tokens (28,000 in, 2,000 out) = 30 million a day. Total about 102 million tokens a day, or 2,244 million a month over 22 working days.

	Frontier API	Mid-tier API	Self-hosted 70B, one server
Monthly cost at 102 M tokens a day	2,244 × ₹500 = ₹11.2 lakh	2,244 × ₹120 = ₹2.7 lakh	₹10 to 14 lakh
Monthly cost at 3 times the volume	₹33.7 lakh	₹8.1 lakh	₹20 to 24 lakh, with a second server
Passes the test set?	Yes	For the assistant, not for summaries	Yes, after prompt tuning
Customer data leaves the network?	Yes, under contract	Yes, under contract	No

Peak load check: 11 million output tokens over a 9-hour day, doubled for the busy hour, is about 680 tokens a second, within one server's planning capacity of around 2,000. The sensible answer is hybrid: loan files hold customer financial data, so they run self-hosted; the assistant uses the mid-tier API on redacted internal content, and moves in-house if volume grows.

> **Costs that are often missed:** long prompts repeated on every call, evaluation runs, a fallback for availability, GPU refresh after four to five years, and the people to run it.

08

Security, data protection and the DPDP Act

Data protection in India

The Digital Personal Data Protection Act, 2023 and its Rules, whose main obligations phase in through 2026 and 2027, apply to personal data in prompts, documents and logs just as to any other system. In practice:

- **Purpose and notice:** personal data in an LLM workflow needs a lawful basis and a stated purpose. Training a model on customer data collected for servicing is a new purpose.
- **Minimisation:** redact personal data the task does not need before it reaches any model, internal or external.
- **Retention and erasure:** prompts and traces containing personal data are personal data. Set retention periods; make erasure reach logs and indexes.
- **Security safeguards and breach reporting:** reasonable safeguards are a legal duty, and breaches must be reported to the Data Protection Board and affected people. CERT-In expects cyber incidents to be reported within 6 hours.
- **Processors:** an API provider processing personal data on your behalf needs a contract that matches your obligations.
- **Sector rules:** RBI, IRDAI and SEBI requirements on outsourcing, data location and audit may be stricter than the Act. Check them per use case.

Threats specific to LLMs

Threat	Control
Prompt injection: hidden instructions in a document or email	Treat retrieved text as data; limit what the model can do; filter outputs; test with attack examples.
Leakage across users	Permissions enforced at retrieval time; no shared conversation memory between users.
Sensitive data in logs	Redact before logging; restrict who can read traces; time-limited retention.
Unapproved models and keys	All traffic through the gateway; outbound access to AI services blocked except via the gateway.
Untrusted model files	Download from known sources, verify checksums, prefer safetensors format, scan containers.

> **A useful reference:** the OWASP Top 10 for LLM Applications lists prompt injection first. Use it as the basis of the security review before go-live.

09

Operating it: evaluation, monitoring and change

An LLM system that worked in March can be worse in June without anyone touching it: the provider updates the model, documents change, users ask new things.

Measure quality all the time

- **Offline:** re-run the test set before every change to model, prompt, retrieval or redaction rules. No change goes live with a lower score without a written reason.
- **Online:** thumbs up or down with an optional comment, plus a weekly sample of 30 to 50 live answers reviewed by people who know the work.
- **Drift:** alert on falling scores, rising complaints, more refusals or a change in answer length after any change.

Watch the platform

Metric	Why it matters	Typical alert
Time to first token, 95th percentile	Users feel this first	Above 2 seconds for chat
GPU memory and cache use	Full cache means queued requests	Above 90% for 15 minutes
Queue length and rejected requests	Capacity is short	Any sustained queue at busy hour
Tokens and cost per team	Budgets and surprises	80% of monthly budget
Error and fallback rate	A model or provider is failing	Above 1% of requests

Change models safely

Keep a catalogue of approved models with version, licence, allowed data classes and test score. A new version passes the test set and a security check, then takes a small share of gateway traffic before full rollout, with a one-step rollback. Pin API model versions where possible.

10

A practical roadmap

A first production use case typically takes 10 to 14 weeks.

Stage	Typical duration	Outcome
1. Triage	2 weeks	Use cases scored, one or two chosen with business owners, data classified, baseline time and cost measured.
2. Test and compare	3 to 4 weeks	Test set of 100 to 300 real cases, two or three models scored on quality, speed and cost, hosting decision.
3. Guarded build	3 to 5 weeks	Gateway with sign-in, redaction, routing, budgets and logging; GPU platform if self-hosting; security review.
4. Pilot	4 weeks	Real users in one department, quality and time saved measured against baseline, issues fixed.
5. Scale and operate	Ongoing	More use cases on the same gateway, monthly quality review, model catalogue and change process in use.

> **Order matters:** build the gateway with the first use case, not after the fifth. Retrofitting it later is slow and rarely complete.

11

Ten common mistakes

1. **No approved option for staff.** Shadow use of public tools continues with company data.
2. **Choosing a model from a leaderboard.** Test on your own examples instead.
3. **Testing only on easy questions.** Real users find the edge cases on day one.
4. **Fine-tuning to teach facts.** Use RAG for anything that changes.
5. **Self-hosting below the crossover without a data reason.** GPUs sit idle and cost more than the API would.
6. **Applications calling models directly.** No central view of usage, cost or data flows.
7. **Sending whole documents on every call.** Retrieve the relevant passages and cache shared prompts.
8. **Logging everything forever.** Logs become the largest store of personal data in the company.
9. **Letting the provider change the model silently.** Pin versions and re-test before switching.
10. **No owner after go-live.** Quality drifts and nobody notices until users stop using it.

12

Enterprise LLM readiness checklist (36 points)

Use this list to score your current position. Anything you cannot tick with evidence is a gap worth closing.

Use cases and ownership

- [ ] An approved LLM option available to staff, with a usage policy
- [ ] Use cases scored on frequency, value, data sensitivity and ease of checking
- [ ] A named business owner for each use case in production
- [ ] Baseline time and cost measured before the pilot
- [ ] Decisions about people kept with people, with the model as support only
- [ ] A shared register of LLM use cases, models and data classes

Data and compliance

- [ ] Data classes defined, with which may go to external APIs
- [ ] Lawful basis and purpose recorded for personal data in each use case
- [ ] Redaction of personal data before any model, including Indian identifiers
- [ ] Retention periods set for prompts, answers and traces
- [ ] Erasure requests reach logs and indexes
- [ ] Provider contracts cover training use, retention, location and breach notice
- [ ] Sector rules (RBI, IRDAI, SEBI or others) checked per use case

Models and evaluation

- [ ] Test set of 100 to 300 real cases per use case, with agreed answers
- [ ] Out-of-scope and hard cases included in the test set
- [ ] At least two models compared on quality, speed and cost
- [ ] Quantised models re-tested before use
- [ ] Model licences reviewed for your use and scale
- [ ] Model catalogue with version, licence, allowed data classes and score

Platform and serving

- [ ] Model servers with continuous batching and prefix caching
- [ ] GPU memory sized for weights, cache and peak concurrency
- [ ] Second replica, second server or API fallback for availability
- [ ] Model files from known sources, checksums verified
- [ ] Busy-hour load tested before go-live

Gateway, cost and security

- [ ] All model traffic through one gateway; direct outbound access blocked
- [ ] Company sign-in for people, one key per team for applications
- [ ] Routing by data class enforced, not advisory
- [ ] Monthly budgets per team with alerts at 50, 80 and 100 percent
- [ ] Caps on tokens per request and rate limits per team
- [ ] Prompt injection tested using the OWASP Top 10 for LLM Applications
- [ ] Logs readable only by named roles

Operations and change

- [ ] Test set re-run before every model, prompt or retrieval change
- [ ] Weekly review of a sample of live answers
- [ ] Alerts on latency, cache use, errors, cost and guardrail hits
- [ ] Gradual rollout and one-step rollback for model changes
- [ ] Quarterly cost review against the crossover point

13

Glossary

Term	Meaning
Token	A piece of a word. Models read, write and are priced in tokens; about 1,300 tokens per 1,000 English words.
Time to first token	How long a user waits before the answer starts to appear.
Continuous batching	Serving technique that adds and removes requests at every step to keep the GPU busy.
KV cache	GPU memory holding the processed context of each active request.
Quantisation	Storing model weights in fewer bits (FP8, 4-bit) to save memory and speed up serving.
RAG	Retrieval-augmented generation: finding relevant passages and giving them to the model to answer from.
LoRA	A fine-tuning method that trains a small set of extra weights instead of the whole model.
LLM gateway	A single entry point for all model traffic that handles access, redaction, routing, budgets and logging.
Prompt injection	Instructions hidden in text the model reads, intended to make it ignore its rules.
Data Fiduciary	Under the DPDP Act, the organisation that decides why and how personal data is processed.

About Vakratron Systems

Vakratron Systems LLP is a vendor-neutral infrastructure design firm, recognised as a startup by DPIIT. We design data centre, disaster recovery, cloud, GPU and AI platforms for enterprises and government buyers. Every design starts from the workload, compares realistic options with their costs, writes its assumptions down, and is tested before handover.

How we can help with enterprise LLMs

- **LLM review:** a two to three week look at your use cases, data rules and current usage, with a plain recommendation on models, hosting and cost.
- **Model comparison:** we build a test set from your real work with your teams and score two or three candidate models on quality, speed and cost.
- **Platform design:** gateway, redaction, budgets, GPU sizing and an operating model, written down so you can take it to vendors or build it yourselves.

Talk to us
connect@vakratronsys.com

Online
vakratronsys.com/advisory

This paper gives general guidance based on common enterprise requirements. Figures are planning ranges, not quotations. Apply them to your environment with care, or ask us to help. © 2026 Vakratron Systems LLP.